DATABASE MANAGEMENT SYSTEMS

ASSIGNMENT 1 – CO1

Design, Analyse and Validate a Storage and Indexing Strategy for the Museum Condition-Report Database

Assignment information	Details
Course	CSA05 – Database Management Systems
Case study	Meridian Museum – Collection Loans and Exhibition Planning
Nature	Individual assignment
Total marks	50
Student name	Jagarajupalli Bhanuprakash Reddy
Register number	192524171
Department / Institution	AI&DS/SIMATS
Faculty	Dr.Shankar Shanmuganathan
Submission date	02-11-2026

ABSTRACT

This assignment develops and evaluates a database design for Meridian Museum, which borrows artifacts for temporary exhibitions. The design separates four facts that must not be conflated: the legal ownership of an artifact, temporary loan custody, exhibition placement, and condition evidence. The preferred model uses a stable PERSON identity with role-based participation, an independently identified LOAN occurrence, LOAN_OBJECT as the associative entity between loans and artifacts, and separately identified condition and placement records. The later shared-ownership requirement is addressed by extending the design with ARTIFACT_OWNER and time-bounded OWNERSHIP_PERIOD records. The report also presents relational mapping, MySQL implementation patterns, deterministic workload generation, operational queries, integrity tests, data-model comparisons, and a migration-validation strategy.

TABLE OF CONTENTS

- 1. Requirements and Alternative Conceptual Designs
- 2. Entities, Attributes and Keys
- 3. Relationships and Structural Constraints
- 4. ER Diagram
- 5. EER Modelling
- 6. Weak Entities and Complex Attributes
- 7. Relational Mapping
- 8. MySQL Implementation and Workload Generation
- 9. Use-Case Query Validation
- 10. Comparison of Data Models
- 11. Final Design Recommendation
- 12. Re-evaluation and Adaptation
- Conclusion
- References and Evidence Notes

CASE STUDY AND DESIGN OBJECTIVES

Meridian Museum maintains 8 collection units, 35 curators, 1,200 lenders and 9,000 catalogued artifacts. Each artifact has one current lender under the original rule set. A loan belongs to one lender and contains multiple artifacts. Display slots are numbered within an exhibition. An artifact may have successive loans, and a curator may also lend a privately owned object. The changed design challenge permits shared legal ownership and changes in ownership shares while an existing loan remains active.

The central design principle is that an artifact is a persistent physical object, while a loan is a time-bounded custody event. An exhibition placement records where an object is displayed, and a condition report records an inspection at a particular time. None of these records should overwrite another.

Design objectives

- Maintain one stable identifier for each person, artifact, loan, report and exhibition.
- Represent one loan containing multiple artifact lines and successive loans for the same artifact.
- Preserve dispatch, in-display and return condition evidence as historical records.
- Identify display slots within their owning exhibitions.
- Support overlapping roles, such as a curator who is also a private lender.
- Support shared, time-varying legal ownership without rewriting custody history.
- Validate integrity rules and evaluate performance at four lender population sizes.

TASK 1 – REQUIREMENTS AND ALTERNATIVE CONCEPTUAL DESIGNS (5 MARKS)

1.1 Operational users and shared information

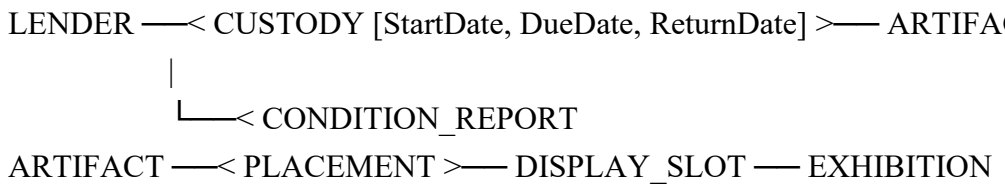
The Registrar creates and closes loan records, checks the lender, agreement number and custody dates, and records return information. The Curator plans exhibitions and assigns artifacts to display slots. The Conservator or Condition Officer records the physical state at dispatch, during display where required, and at return. The Collection Manager maintains artifact identity and classification. All operational users must share ArtifactID and LoanID so that the same physical object and custody occurrence are referenced consistently.

User	Records maintained	Information shared
Registrar	LOAN, LOAN_OBJECT, return details	Lender identity, agreement, start/due/return dates, artifact lines
Curator	EXHIBITION, DISPLAY_SLOT, PLACEMENT	Exhibition period, slot, artifact and placement interval
Condition Officer	CONDITION_REPORT and observations	Artifact, relevant loan line, inspection time, result and evidence
Collection Manager	ARTIFACT and collection classifications	Accession number, title, collection unit and type

1.2 Alternative A – Dated relationship model

Alternative A models borrowing as a dated relationship between LENDER and ARTIFACT. StartDate, DueDate, ReturnDate and agreement details are attributes of the relationship. Condition evidence is attached to the dated

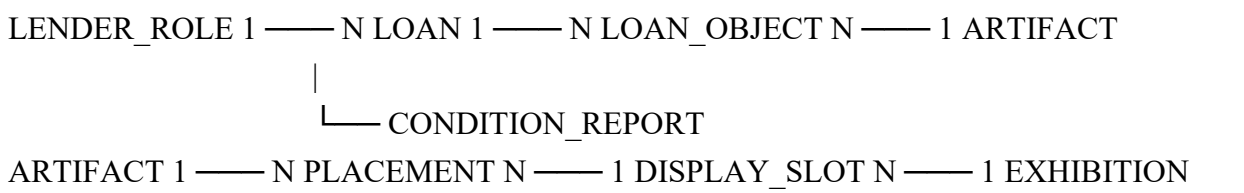
custody relationship. Repeated loans can be distinguished by including the date or a generated relationship identifier.



This model resolves the case where the same lender borrows the same artifact in two different periods: a report from the first return must not be mistaken for a report from the later loan. Its weakness is that a single agreement containing several artifacts is not represented naturally as one independently addressable event. Agreement-level facts may be repeated across relationship instances.

1.3 Alternative B – Independently identified LOAN occurrence

Alternative B introduces LOAN as a strong entity identified by LoanID. One lender may be responsible for many loans, but each loan has exactly one responsible lender under the original rules. LOAN_OBJECT is an associative entity connecting LOAN and ARTIFACT. It stores one row for each artifact line in a loan. CONDITION_REPORT refers to the physical artifact and, where custody-specific, the corresponding loan line.

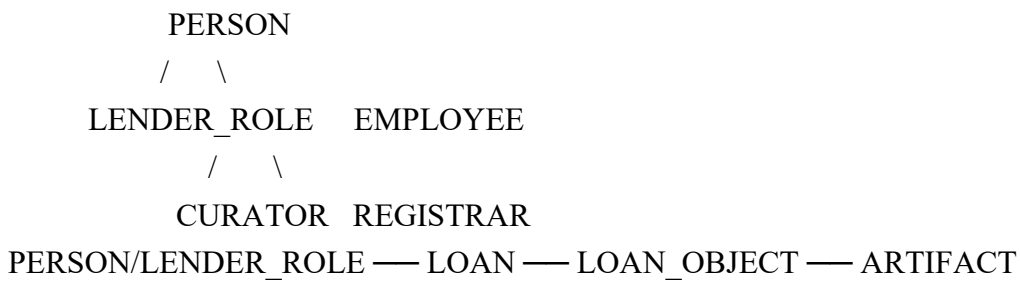


The stable LoanID preserves a loan as one operational event even when it contains several artifacts. The same ArtifactID can appear in multiple LOAN_OBJECT rows over time, preserving successive custody periods. The model avoids overwriting earlier custody records.

1.4 Alternative C – Shared PERSON identity with roles

Alternative C addresses identity overlap. A single PERSON entity stores common identity and contact details. LENDER_ROLE and EMPLOYEE are roles of PERSON; EMPLOYEE may be specialised into CURATOR and

REGISTRAR. The same person can hold more than one role. The loan-occurrence structure from Alternative B is retained, but role records refer to the shared PersonID.



If a curator lends a privately owned object, the museum stores one person identity with both employee/curator and lender roles. Contact information is not duplicated. The trade-off is the need to enforce subtype and role constraints and to use joins to retrieve role-specific information.

1.5 Comparison and selection

Alternative	Main distinction	Case problem addressed	Limitation
A – Dated relationship	Custody dates belong to lender–artifact relationship	Distinguishes repeated custody periods	Multi-artifact loan agreement is awkward to model as one event
B – LOAN entity	Each loan has independent LoanID	One agreement, many artifacts, successive loans and traceable evidence	Requires LOAN_OBJECT and additional joins
C – PERSON roles	One identity may hold several roles	Avoids duplicate identity when curator is also lender	Role constraints and subtype joins add complexity

The preferred original design combines B and C. It uses an independently identified LOAN occurrence, LOAN_OBJECT for loan lines, and shared

PERSON identity with role-based participation. The alternatives are materially different: A places event facts on a relationship; B creates an independently addressable custody event; C changes the identity structure. Merely renaming LOAN would not create these distinctions.

Ownership remains separate from custody. Under the original rule, one current lender is associated with an artifact, but the changed requirement permits several legal owners. Task 12 therefore extends the model with ownership history rather than changing historical LOAN records.

TASK 2 – ENTITIES, ATTRIBUTES AND KEYS (2 MARKS)

2.1 Entity and key catalogue

Entity	**Important attributes**	**Primary / candidate key**
PERSON	PersonID, GivenName, MiddleName, FamilyName, Email, Address	PK PersonID; Email candidate only if verified unique
LENDER_ROLE	PersonID, LenderSince, LenderStatus	PK/FK PersonID
EMPLOYEE	PersonID, EmployeeNo, HireDate, Role	PK/FK PersonID; EmployeeNo UNIQUE
CURATOR	PersonID, Specialisation	PK/FK PersonID
REGISTRAR	PersonID, AuthorisationLevel	PK/FK PersonID
ARTIFACT	ArtifactID, AccessionNo, Title, CollectionUnitID, CollectionTypeID	PK ArtifactID; AccessionNo UNIQUE

LOAN	LoanID, LenderPersonID, AgreementNo, StartDate, DueDate, ReturnDate	PK LoanID; AgreementNo UNIQUE if policy guarantees it
LOAN_OBJECT	LoanID, ArtifactID, LineNo, LoanValue, DispatchCondition	PK (LoanID, ArtifactID); UNIQUE (LoanID, LineNo)
EXHIBITION	ExhibitionID, Name, StartDate, EndDate	PK ExhibitionID
DISPLAY_SLOT	ExhibitionID, SlotNo, LocationLabel	PK (ExhibitionID, SlotNo)
PLACEMENT	PlacementID, ArtifactID, ExhibitionID, SlotNo, FromDate, ToDate	PK PlacementID
CONDITION_REPORT	ReportID, LoanID, ArtifactID, InspectedAt, ReportType, Notes	PK ReportID
COLLECTION_UNIT	CollectionUnitID, UnitName	PK CollectionUnitID
COLLECTION_TYPE	CollectionTypeID, TypeName, Description	PK CollectionTypeID
PERSON_PHONE	PersonID, PhoneNumber, PhoneType	PK (PersonID, PhoneNumber)

2.2 Strong and weak entity identity

PERSON, ARTIFACT, LOAN, EXHIBITION, CONDITION_REPORT and COLLECTION_UNIT have identities independent of a particular loan. DISPLAY_SLOT is existence-dependent on EXHIBITION. Slot number 4 in

one exhibition is not the same slot as slot number 4 in another exhibition. DISPLAY_SLOT is therefore a weak entity with partial key SlotNo and identifying owner ExhibitionID. Its full key is (ExhibitionID, SlotNo).

2.3 Attribute classification

Attribute / concept	Classification	Reason
Person name	Composite	Stored as GivenName, MiddleName and FamilyName
Telephone numbers	Multivalued	A person can have several numbers; store one row per number
StartDate, DueDate, ReturnDate	Stored simple date facts	They record operational events and must remain historically accurate
Loan duration / overdue days	Derived	Calculated from custody dates and current date
Latest condition summary	Derived from evidence	Calculated from timestamped reports; original reports remain stored
CollectionType	Coded/reference attribute	Separate entity is useful when code, description and rules exist

An exhibition name cannot identify a display slot because names can repeat, change or contain spelling variations. A slot number is unique only within its exhibition. Surrogate identifiers such as PersonID, ArtifactID and LoanID remain stable even when names change. AUTO_INCREMENT or UUID generation, together with UNIQUE constraints on business keys, prevents duplicate identifiers during concurrent registration.

TASK 3 – RELATIONSHIPS AND STRUCTURAL CONSTRAINTS (6 MARKS)

3.1 Binary relationships

Relationship	Cardinality	Participation and justification
LENDER_ROLE – LOAN	1 : N	Each loan has exactly one responsible lender; a lender may have zero or many loans
LOAN – LOAN_OBJECT	1 : N	A loan should contain one or more lines when confirmed; a draft loan may temporarily have none
ARTIFACT – LOAN_OBJECT	1 : N	An artifact may have zero or many historical loan lines
EXHIBITION – DISPLAY_SLOT	1 : N	Each slot belongs to exactly one exhibition; an exhibition may have multiple slots
ARTIFACT – CONDITION_REPORT	1 : N	An artifact may have many reports; a report concerns one artifact
ARTIFACT – PLACEMENT	1 : N	An artifact may have multiple placement periods over time

3.2 Binary versus ternary association

The normal operational relationships are binary because each fact can be stated between two entities: a lender is responsible for a loan; a loan contains

an artifact line; a curator manages an exhibition; and a placement assigns an artifact to a slot. A possible ternary relationship LENDER–CURATOR–EXHIBITION could record which lender, curator and exhibition participated in a particular coordination event. It should only be introduced if the business fact genuinely depends on all three participants simultaneously.

Counterexample: if Lender L1 supplies an artifact for Exhibition E1 and Curator C1 coordinates the loan, a three-way coordination tuple (L1, C1, E1) may be required. Separate binary projections can lose the fact that these three values occurred together. If the museum only needs to know which curator manages an exhibition and which lender is responsible for a loan, the independent binary relationships are sufficient and the ternary relation is unnecessary.

3.3 Minimum and maximum participation

A lender may exist before making a loan (0..N); every confirmed loan must reference exactly one lender (1..1). A loan should have at least one LOAN_OBJECT line (1..N) at confirmation, although a draft may temporarily have zero. An artifact may never have been loaned (0..N loan lines). Every DISPLAY_SLOT must belong to one exhibition (1..1), while an exhibition may have zero or many slots (0..N).

3.4 Placement-to-slot constraint

PLACEMENT must reference the composite key (ExhibitionID, SlotNo) of DISPLAY_SLOT. This prevents a placement from referring to slot 7 in an exhibition where slot 7 does not exist. The pair is a composite foreign key, not two unrelated foreign keys.

3.5 Facts belong on the correct relation

- Loan-level facts: AgreementNo, LenderPersonID, StartDate, DueDate, ReturnDate and RegistrarPersonID belong on LOAN.
- Loan-line facts: LineNo, LoanValue and DispatchCondition belong on LOAN_OBJECT because they can differ by artifact within one loan.

- Inspection facts: InspectedAt, ReportType, Notes and EvidenceURI belong on CONDITION_REPORT.
- Placement facts: FromDate and ToDate belong on PLACEMENT, not on ARTIFACT or LOAN.

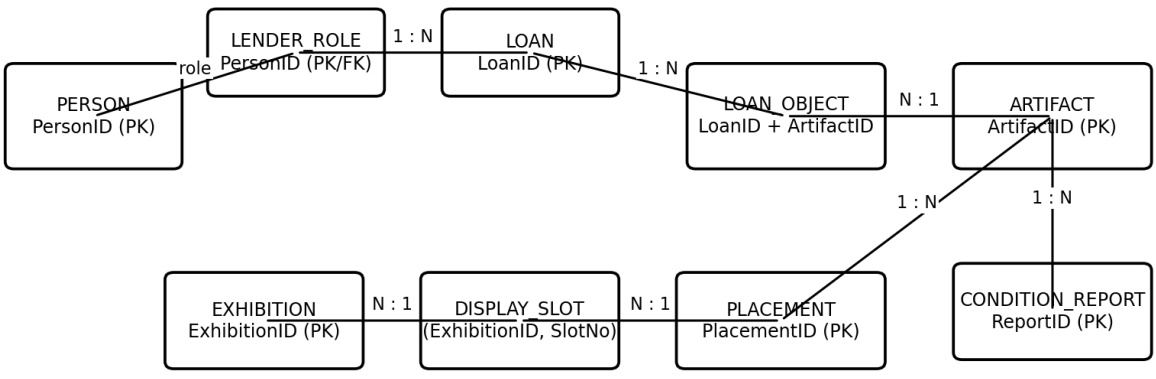
3.6 Ternary decomposition test

Suppose the ternary relation Coordination(Lender, Curator, Exhibition) contains (L1,C1,E1) and (L2,C1,E2). Its binary projections contain Lender–Curator pairs (L1,C1), (L2,C1), Lender–Exhibition pairs (L1,E1), (L2,E2), and Curator–Exhibition pairs (C1,E1), (C1,E2). Joining these projections can produce spurious combinations (L1,C1,E2) and (L2,C1,E1). Therefore the binary projections do not reconstruct the original ternary relation without extra tuples. Keep a ternary association only if the case needs that joint fact.

TASK 4 – ER DIAGRAM (4 MARKS)

The following ER representation shows the preferred original model. LOAN_OBJECT resolves the many-to-many relationship between LOAN and ARTIFACT. CONDITION_REPORT retains the inspection evidence and links it to the loan line when custody-specific. PLACEMENT connects the physical artifact to a display slot independently of the loan.

Preferred conceptual ER model



Custody, condition evidence and exhibition placement are separate relationships.

Figure 1. Preferred ER model for loan custody, condition evidence and exhibition placement.

4.1 Key relationship interpretation

- One LENDER_ROLE may be responsible for many LOAN records; each LOAN has one responsible lender.
- One LOAN has multiple LOAN_OBJECT lines; each line identifies one artifact within that loan.
- One ARTIFACT may appear in many loan lines across successive custody periods.
- One ARTIFACT may have multiple CONDITION_REPORT records over time.
- One EXHIBITION owns many DISPLAY_SLOT records; each slot is identified by ExhibitionID and SlotNo.
- One ARTIFACT may have multiple PLACEMENT periods, each linked to a valid slot.

4.2 Why ownership is not shown as custody

The original ER model records the party responsible for the loan, not necessarily every legal owner. The changed requirement adds ownership history in Task 12. Keeping the two structures separate prevents a change in title from rewriting a loan agreement or condition report.

TASK 5 – EER MODELLING (3 MARKS)

5.1 PERSON specialisation

PERSON is the supertype containing common attributes such as PersonID, names, email and address. LENDER_ROLE and EMPLOYEE are role subtypes. EMPLOYEE may be specialised into CURATOR and REGISTRAR. The specialisation is overlapping because one employee may hold more than one employee role, and a PERSON may also have a lender

role. The model should not assume that lender and employee identities are mutually exclusive.

```
PERSON(PersonID, GivenName, MiddleName, FamilyName, Email, Address)
  ├── LENDER_ROLE(PersonID, LenderSince, LenderStatus)
  └── EMPLOYEE(PersonID, EmployeeNo, HireDate)
      ├── CURATOR(PersonID, Specialisation)
      └── REGISTRAR(PersonID, AuthorisationLevel)
```

5.2 Person and organisation ownership category

For the changed requirement, an artifact owner may be an individual or an organisation. A category ARTIFACT_OWNER can represent the union of PERSON and ORGANIZATION. A practical relational implementation uses a common OwnerID with a type discriminator and subtype-specific tables, or uses separate nullable PersonID and OrganizationID foreign keys with a CHECK requiring exactly one to be populated. The subtype approach avoids forcing organisation details into person columns.

5.3 Why a wide table is unsuitable

A single wide table with lender, curator and registrar columns mixes identity with roles, creates many NULL values and duplicates people who hold multiple roles. A supertype/subtype model stores common attributes once and role-specific attributes only where needed. This supports a curator who also lends a privately owned artifact.

TASK 6 – WEAK ENTITIES AND COMPLEX ATTRIBUTES (3 MARKS)

6.1 DISPLAY_SLOT as a weak entity

DISPLAY_SLOT depends on EXHIBITION for its identity. SlotNo is only a partial key; the complete key is (ExhibitionID, SlotNo). The identifying relationship is mandatory because a slot without its exhibition has no meaning. An ordinary foreign key alone does not make an entity weak; the owner-dependent key is the defining feature.

6.2 Multivalued and composite attributes

Telephone numbers are stored in PERSON_PHONE as one row per number, with composite key (PersonID, PhoneNumber). This supports searching, validating and updating an individual number without parsing comma-separated text. PersonName is composite and is stored in GivenName, MiddleName and FamilyName. A formatted display name can be derived for presentation.

6.3 Derived attributes and coded concepts

Concept	Storage decision	Reason
Loan duration	Derived from StartDate and ReturnDate (or current date if open)	Avoids stale calculated values
Overdue days	Derived from DueDate and current date for open loans	Changes as time passes
Current custody status	Derived from dates or maintained as controlled state	Must agree with custody dates
Latest condition summary	Derived from latest condition observations	Preserves full report history
Condition history	Store every report and observation	Historical evidence must not be overwritten
CollectionType	Separate reference entity	Has stable code, label and description
ReportType	Reference entity when controlled vocabulary is maintained	Avoids inconsistent free-text categories

ArtifactValue is the museum's estimated or insured value of the physical object. LoanValue is the valuation recorded for a particular loan line and may reflect the agreement or insurance valuation at that time. A later

valuation change must not rewrite a historical loan valuation. If values change over time, dated valuation records should be stored.

TASK 7 – MAP THE EER MODEL TO A RELATIONAL SCHEMA (4 MARKS)

7.1 Relational schema catalogue

PERSON(PersonID PK, GivenName, MiddleName, FamilyName, Email, Address)
PERSON_PHONE(PersonID PK/FK -> PERSON, PhoneNumber PK, PhoneType)
LENDER_ROLE(PersonID PK/FK -> PERSON, LenderSince, LenderStatus)
EMPLOYEE(PersonID PK/FK -> PERSON, EmployeeNo UNIQUE, HireDate)
CURATOR(PersonID PK/FK -> EMPLOYEE, Specialisation)
REGISTRAR(PersonID PK/FK -> EMPLOYEE, AuthorisationLevel)
COLLECTION_UNIT(CollectionUnitID PK, UnitName UNIQUE)
COLLECTION_TYPE(CollectionTypeID PK, TypeName UNIQUE, Description)
ARTIFACT(ArtifactID PK, AccessionNo UNIQUE, Title, Description,
CollectionUnitID FK, CollectionTypeID FK, ArtifactValue)
LOAN(LoanID PK, LenderPersonID FK -> LENDER_ROLE, AgreementNo UNIQUE,
StartDate, DueDate, ReturnDate, RegistrarPersonID FK -> REGISTRAR)
LOAN_OBJECT(LoanID PK/FK -> LOAN, ArtifactID PK/FK -> ARTIFACT,
LineNo, LoanValue, DispatchCondition)
EXHIBITION(ExhibitionID PK, Name, StartDate, EndDate)
DISPLAY_SLOT(ExhibitionID PK/FK -> EXHIBITION, SlotNo PK, LocationLabel)
PLACEMENT(PlacementID PK, ArtifactID FK, ExhibitionID, SlotNo,
FromDate, ToDate, FK(ExhibitionID, SlotNo) -> DISPLAY_SLOT)
CONDITION_REPORT(ReportID PK, LoanID, ArtifactID, InspectorPersonID FK,
InspectedAt, ReportType, Notes, EvidenceURI,
FK(LoanID, ArtifactID) -> LOAN_OBJECT)

7.2 Mapping decisions

- Each strong entity becomes a relation with its primary key.
- The weak DISPLAY_SLOT relation includes the owner's ExhibitionID in its composite primary key.
- The 1:N lender-to-loan relationship is mapped by placing LenderPersonID in LOAN.

- The M:N LOAN–ARTIFACT relationship becomes LOAN_OBJECT.
- The multivalued phone attribute becomes PERSON_PHONE.
- Overlapping specialisation is mapped with a PERSON supertype and one table per subtype.
- CONDITION_REPORT has a composite foreign key to LOAN_OBJECT so that a loan-specific report cannot reference an artifact absent from that loan.

7.3 Deletion behaviour and indexes

Historical LOAN, LOAN_OBJECT and CONDITION_REPORT rows should normally be retained. RESTRICT is appropriate for deleting a person or artifact referenced by historical records. Operational retirement should use status flags or soft deletion. Avoid cascading deletion of custody and condition history. Useful indexes include LOAN(LenderPersonID, StartDate), LOAN(DueDate, ReturnDate), LOAN_OBJECT(ArtifactID), CONDITION_REPORT(ArtifactID, InspectedAt) and PLACEMENT(ArtifactID, FromDate, ToDate).

TASK 8 – IMPLEMENT THE SCHEMA AND POPULATE INSTANCES (4 MARKS)

8.1 Database creation and core DDL

Run the implementation in a dedicated test database. The following DDL illustrates the core schema and uses MySQL-compatible constraints. Execute parent tables before child tables. The sample script is a starting implementation; the complete reproducible workload-generation process follows.

```
CREATE DATABASE IF NOT EXISTS meridian_museum;
USE meridian_museum;

CREATE TABLE Person (
  PersonID BIGINT PRIMARY KEY AUTO_INCREMENT,
  GivenName VARCHAR(60) NOT NULL,
```

```
  MiddleName VARCHAR(60),
  FamilyName VARCHAR(60) NOT NULL,
  Email VARCHAR(150) UNIQUE,
  Address VARCHAR(255)
);

CREATE TABLE PersonPhone (
  PersonID BIGINT NOT NULL,
  PhoneNumber VARCHAR(25) NOT NULL,
  PhoneType VARCHAR(20) NOT NULL,
  PRIMARY KEY (PersonID, PhoneNumber),
  FOREIGN KEY (PersonID) REFERENCES Person(PersonID)
    ON UPDATE CASCADE ON DELETE RESTRICT
);

CREATE TABLE LenderRole (
  PersonID BIGINT PRIMARY KEY,
  LenderSince DATE,
  LenderStatus VARCHAR(20) NOT NULL DEFAULT 'ACTIVE',
  FOREIGN KEY (PersonID) REFERENCES Person(PersonID)
    ON UPDATE CASCADE ON DELETE RESTRICT,
  CHECK (LenderStatus IN ('ACTIVE','INACTIVE'))
);

CREATE TABLE Employee (
  PersonID BIGINT PRIMARY KEY,
  EmployeeNo VARCHAR(30) NOT NULL UNIQUE,
  HireDate DATE NOT NULL,
  FOREIGN KEY (PersonID) REFERENCES Person(PersonID)
    ON UPDATE CASCADE ON DELETE RESTRICT
);

CREATE TABLE Curator (
  PersonID BIGINT PRIMARY KEY,
  Specialisation VARCHAR(100),
  FOREIGN KEY (PersonID) REFERENCES Employee(PersonID)
    ON UPDATE CASCADE ON DELETE RESTRICT
);
```

```
CREATE TABLE Registrar (
  PersonID BIGINT PRIMARY KEY,
  AuthorisationLevel VARCHAR(30),
  FOREIGN KEY (PersonID) REFERENCES Employee(PersonID)
    ON UPDATE CASCADE ON DELETE RESTRICT
);

CREATE TABLE CollectionUnit (
  CollectionUnitID INT PRIMARY KEY AUTO_INCREMENT,
  UnitName VARCHAR(100) NOT NULL UNIQUE
);

CREATE TABLE CollectionType (
  CollectionTypeID INT PRIMARY KEY AUTO_INCREMENT,
  TypeName VARCHAR(80) NOT NULL UNIQUE,
  Description VARCHAR(255)
);

CREATE TABLE Artifact (
  ArtifactID BIGINT PRIMARY KEY AUTO_INCREMENT,
  AccessionNo VARCHAR(40) NOT NULL UNIQUE,
  Title VARCHAR(180) NOT NULL,
  Description TEXT,
  CollectionUnitID INT NOT NULL,
  CollectionTypeID INT NOT NULL,
  ArtifactValue DECIMAL(14,2),
  FOREIGN KEY (CollectionUnitID) REFERENCES CollectionUnit(CollectionUnitID),
  FOREIGN KEY (CollectionTypeID) REFERENCES CollectionType(CollectionTypeID),
  CHECK (ArtifactValue IS NULL OR ArtifactValue >= 0)
);

CREATE TABLE Loan (
  LoanID BIGINT PRIMARY KEY AUTO_INCREMENT,
  LenderPersonID BIGINT NOT NULL,
  AgreementNo VARCHAR(50) NOT NULL UNIQUE,
  StartDate DATE NOT NULL,
  DueDate DATE NOT NULL,
```

```
  ReturnDate DATE,
  RegistrarPersonID BIGINT NOT NULL,
  FOREIGN KEY (LenderPersonID) REFERENCES LenderRole(PersonID),
  FOREIGN KEY (RegistrarPersonID) REFERENCES Registrar(PersonID),
  CHECK (DueDate >= StartDate),
  CHECK (ReturnDate IS NULL OR ReturnDate >= StartDate)
);

CREATE TABLE LoanObject (
  LoanID BIGINT NOT NULL,
  ArtifactID BIGINT NOT NULL,
  LineNo INT NOT NULL,
  LoanValue DECIMAL(14,2),
  DispatchCondition VARCHAR(100),
  PRIMARY KEY (LoanID, ArtifactID),
  UNIQUE (LoanID, LineNo),
  FOREIGN KEY (LoanID) REFERENCES Loan(LoanID) ON DELETE RESTRICT,
  FOREIGN KEY (ArtifactID) REFERENCES Artifact(ArtifactID) ON DELETE RESTRICT,
  CHECK (LineNo > 0),
  CHECK (LoanValue IS NULL OR LoanValue >= 0)
);

CREATE TABLE Exhibition (
  ExhibitionID INT PRIMARY KEY AUTO_INCREMENT,
  Name VARCHAR(150) NOT NULL,
  StartDate DATE NOT NULL,
  EndDate DATE NOT NULL,
  CHECK (EndDate >= StartDate)
);

CREATE TABLE DisplaySlot (
  ExhibitionID INT NOT NULL,
  SlotNo INT NOT NULL,
  LocationLabel VARCHAR(100),
  PRIMARY KEY (ExhibitionID, SlotNo),
  FOREIGN KEY (ExhibitionID) REFERENCES Exhibition(ExhibitionID)
    ON DELETE RESTRICT,
  CHECK (SlotNo > 0)
```

```
);

CREATE TABLE Placement (
  PlacementID BIGINT PRIMARY KEY AUTO_INCREMENT,
  ArtifactID BIGINT NOT NULL,
  ExhibitionID INT NOT NULL,
  SlotNo INT NOT NULL,
  FromDate DATE NOT NULL,
  ToDate DATE,
  FOREIGN KEY (ArtifactID) REFERENCES Artifact(ArtifactID),
  FOREIGN KEY (ExhibitionID, SlotNo)
    REFERENCES DisplaySlot(ExhibitionID, SlotNo),
  CHECK (ToDate IS NULL OR ToDate >= FromDate)
);

CREATE TABLE ConditionReport (
  ReportID BIGINT PRIMARY KEY AUTO_INCREMENT,
  LoanID BIGINT NOT NULL,
  ArtifactID BIGINT NOT NULL,
  InspectorPersonID BIGINT NOT NULL,
  InspectedAt DATETIME NOT NULL,
  ReportType VARCHAR(30) NOT NULL,
  Notes TEXT,
  EvidenceURI VARCHAR(500),
  FOREIGN KEY (LoanID, ArtifactID)
    REFERENCES LoanObject(LoanID, ArtifactID) ON DELETE RESTRICT,
  FOREIGN KEY (InspectorPersonID) REFERENCES Person(PersonID),
  CHECK (ReportType IN ('DISPATCH','IN_DISPLAY','RETURN','OTHER'))
);
```

8.2 Reproducible workload generation

The assignment specifies a deterministic seed of 4211 and four lender population sizes: 100, 1,000, 10,000 and 100,000. A reproducible generator must create parent PERSON rows first, then LENDER_ROLE rows, and then linked LOAN, LOAN_OBJECT and CONDITION_REPORT records. Foreign keys must be selected from rows that already exist; arbitrary random foreign-key values are not safe.

The following MySQL 8 approach uses a recursive sequence for each population. For large loads, a permanent numbers table or a client-side bulk loader is usually more efficient than a very large recursive CTE. Use a fixed seed in the client-side generator and document the exact key-selection rule.

```
-- MySQL 8.0+ example: create a sequence table once
CREATE TABLE IF NOT EXISTS Seq (
  n INT PRIMARY KEY
);

-- Example sequence population; generate the required range in batches
INSERT IGNORE INTO Seq(n)
WITH RECURSIVE numbers AS (
  SELECT 1
  UNION ALL
  SELECT n + 1 FROM numbers WHERE n < 100000
)
SELECT n FROM numbers;

-- Populate a chosen workload size after clearing only the test dataset.
-- :N is a client-side parameter set to 100, 1000, 10000 or 100000.
-- The generator must preserve a deterministic mapping from sequence n to keys.
-- Example pattern:
INSERT INTO Person(GivenName, FamilyName, Email)
SELECT CONCAT('Lender', n), CONCAT('Family', n),
      CONCAT('lender', n, '@example.test')
FROM Seq
WHERE n <= 100000;

-- Create the matching lender role rows using generated PersonID keys.
INSERT INTO LenderRole(PersonID, LenderSince, LenderStatus)
SELECT PersonID, DATE_ADD('2020-01-01', INTERVAL (PersonID MOD 1500) DAY),
'ACTIVE'
FROM Person
WHERE Email LIKE 'lender%@example.test';
```

A practical scale rule is one loan per ten lenders and two loan lines per loan, subject to available artifact and registrar parent rows. The exact number of

artifacts, loan lines and reports should be recorded after loading. The sample values are a workload design, not evidence that a particular execution has already occurred.

8.3 Row-level checks and cross-row rules

CHECK constraints validate values within one row, such as DueDate not preceding StartDate and non-negative LoanValue. They cannot reliably enforce that an artifact has no overlapping active loan, that every confirmed loan has at least one LOAN_OBJECT line, that placement intervals do not overlap for an artifact, or that an inspector is authorised. These require transaction-level validation, triggers, stored procedures or application logic with appropriate locking.

8.4 Invalid-write tests

```
-- Expected rejection: duplicate accession number
INSERT INTO Artifact(AccessionNo, Title, CollectionUnitID, CollectionTypeID)
VALUES ('MM-0001', 'Duplicate test', 1, 1);

-- Expected rejection: due date earlier than start date
INSERT INTO Loan(LenderPersonID, AgreementNo, StartDate, DueDate, RegistrarPersonID)
VALUES (1, 'BAD-DATE', '2026-05-10', '2026-05-01', 3);

-- Expected rejection: unknown collection type
INSERT INTO Artifact(AccessionNo, Title, CollectionUnitID, CollectionTypeID)
VALUES ('MM-9999', 'Invalid type', 1, 9999);

-- Expected rejection: negative loan valuation
INSERT INTO LoanObject(LoanID, ArtifactID, LineNo, LoanValue)
VALUES (1, 1, 99, -500.00);
```

The inserts above are intentional negative tests. Run each separately and record the actual error number and message from the installed MySQL server. Error wording may vary by version and constraint name. Do not claim an unexecuted error message as a genuine result.

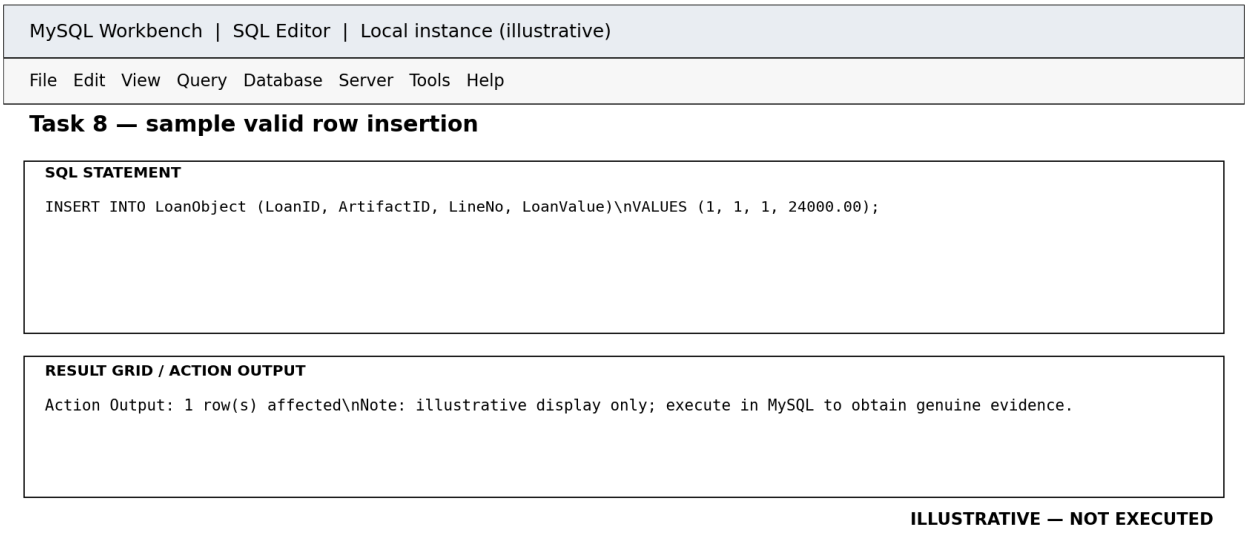


Figure 2. Illustrative Workbench-style insert output; not an actual execution screenshot.

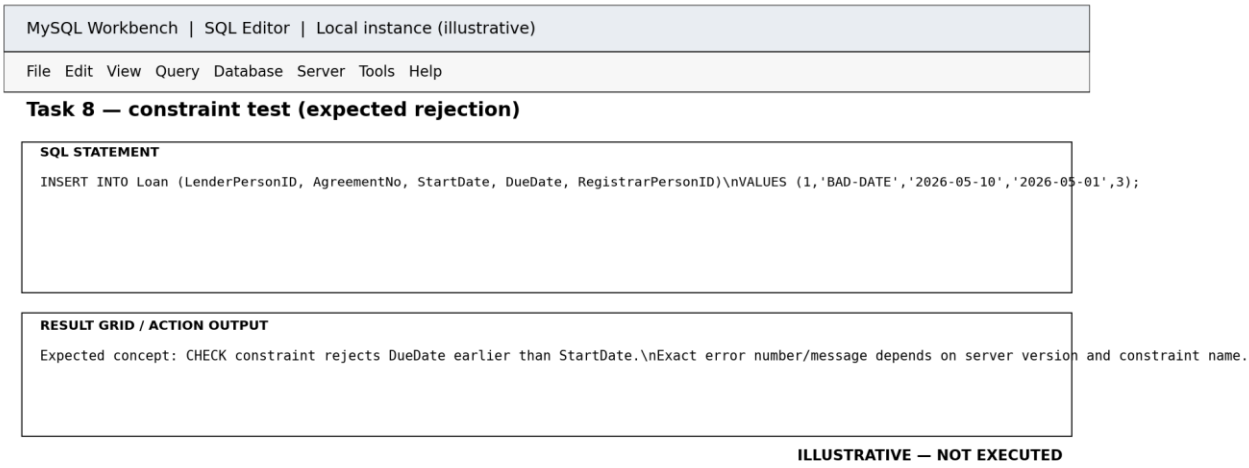


Figure 3. Illustrative Workbench-style constraint rejection; exact server message must be captured during execution.

TASK 9 – VALIDATE THE DESIGN WITH USE-CASE QUERIES (3 MARKS)

9.1 Eight operational queries

The queries test overdue custody, missing condition evidence, loan composition, custody duration, latest inspection, loan-line valuation,

exhibition placement and evidence counts. They are written for the relational schema in this report.

Q1. Open overdue loans

```
SELECT LoanID, AgreementNo, LenderPersonID, DueDate
FROM Loan
WHERE ReturnDate IS NULL
  AND DueDate < CURRENT_DATE
ORDER BY DueDate;
```

Q2. Artifacts without condition evidence

```
SELECT a.ArtifactID, a.AccessionNo, a.Title
FROM Artifact AS a
LEFT JOIN ConditionReport AS cr
  ON cr.ArtifactID = a.ArtifactID
WHERE cr.ReportID IS NULL
ORDER BY a.ArtifactID;
```

Q3. Artifacts included in each loan

```
SELECT l.LoanID, l.AgreementNo, a.ArtifactID, a.Title, lo.LineNo
FROM Loan AS l
JOIN LoanObject AS lo ON lo.LoanID = l.LoanID
JOIN Artifact AS a ON a.ArtifactID = lo.ArtifactID
ORDER BY l.LoanID, lo.LineNo;
```

Q4. Loan custody period and duration

```
SELECT LoanID, StartDate, DueDate, ReturnDate,
      DATEDIFF(COALESCE(ReturnDate, CURRENT_DATE), StartDate) AS CustodyDays
FROM Loan
ORDER BY StartDate;
```

Q5. Latest condition report for each artifact

```
SELECT cr.ArtifactID, cr.ReportID, cr.InspectedAt, cr.ReportType, cr.Notes
FROM ConditionReport AS cr
JOIN (
  SELECT ArtifactID, MAX(InspectedAt) AS LatestAt
  FROM ConditionReport
  GROUP BY ArtifactID
) AS latest
```

```
  ON latest.ArtifactID = cr.ArtifactID
 AND latest.LatestAt = cr.InspectedAt
ORDER BY cr.ArtifactID;
```

Q6. Loan lines and valuations

```
SELECT LoanID, ArtifactID, LineNo, LoanValue, DispatchCondition
FROM LoanObject
ORDER BY LoanID, LineNo;
```

Q7. Exhibition slot placements

```
SELECT e.Name AS Exhibition, p.SlotNo, a.AccessionNo, a.Title,
       p.FromDate, p.ToDate
FROM Placement AS p
JOIN Exhibition AS e ON e.ExhibitionID = p.ExhibitionID
JOIN Artifact AS a ON a.ArtifactID = p.ArtifactID
ORDER BY e.ExhibitionID, p.SlotNo;
```

Q8. Condition evidence count by artifact

```
SELECT a.ArtifactID, a.AccessionNo, a.Title,
       COUNT(cr.ReportID) AS ReportCount
FROM Artifact AS a
LEFT JOIN ConditionReport AS cr
  ON cr.ArtifactID = a.ArtifactID
GROUP BY a.ArtifactID, a.AccessionNo, a.Title
ORDER BY ReportCount, a.ArtifactID;
```

Q2 uses LEFT JOIN so artifacts with no matching condition report remain visible. If every artifact has a report, the correct result is zero rows; an empty result is meaningful and must be recorded rather than omitted.

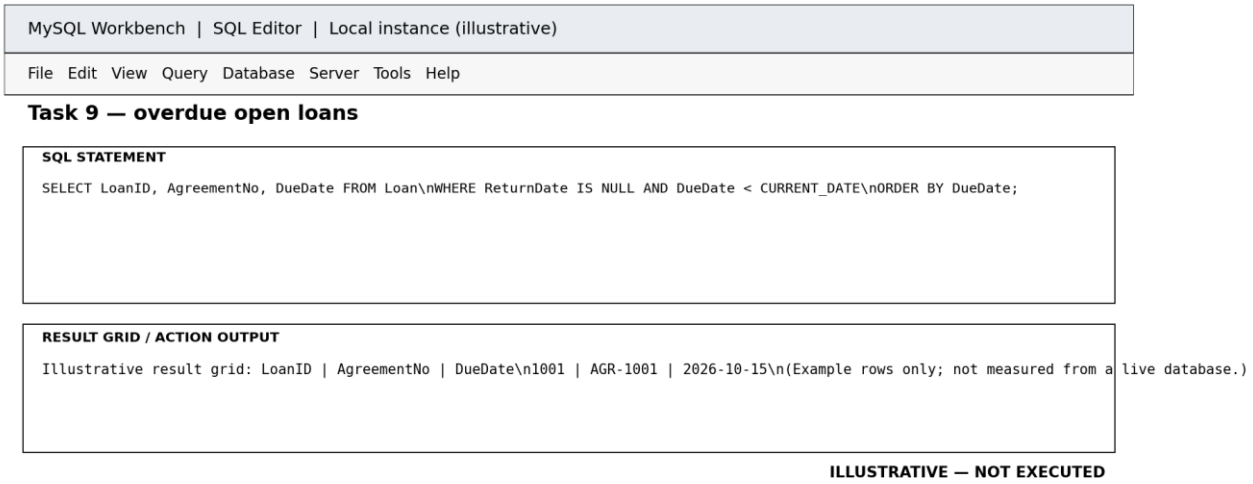


Figure 4. Illustrative Workbench-style result grid for Q1; sample rows are not live data.

9.2 Performance measurement protocol

For each workload size, execute each query five times under the same machine, server configuration, index set and cache procedure. Record elapsed milliseconds, sort the five values and report the middle value as the median. Record result counts and the number of explicit JOIN operations. Do not fabricate elapsed times or imply that sample numbers were measured.

Lender rows	Q1 result count	Q2 result count	Q3 result count	Median elapsed time
100	Record after execution	Record after execution	Record after execution	Measure 5 runs
1,000	Record after execution	Record after execution	Record after execution	Measure 5 runs
10,000	Record after execution	Record after execution	Record after execution	Measure 5 runs
100,000	Record after execution	Record after execution	Record after execution	Measure 5 runs

9.3 Flat-file anomalies

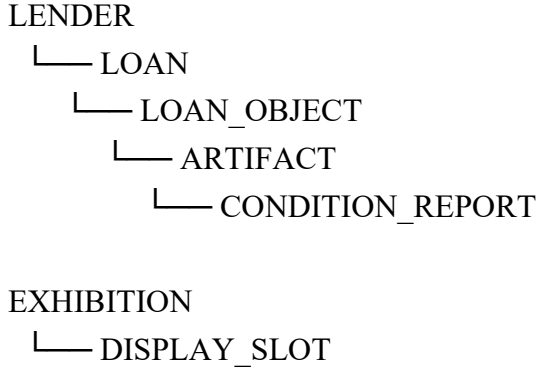
A flat file such as (LoanID, AgreementNo, LenderName, LenderPhone, ArtifactID, ArtifactTitle, DueDate, ConditionNotes) repeats lender and loan details for every artifact line. An insertion anomaly occurs when a lender cannot be registered without inventing a loan row. An update anomaly occurs when a phone-number change must be repeated in many rows, and missed updates create inconsistency. A deletion anomaly occurs when deleting the only artifact line also removes the only copy of the loan or lender information. Separating PERSON/LENDER_ROLE, LOAN, ARTIFACT, LOAN_OBJECT and CONDITION_REPORT removes these anomalies.

TASK 10 – COMPARE FILE-BASED, HIERARCHICAL AND NETWORK MODELS (4 MARKS)

10.1 File-based model

A file-based implementation may keep separate lender, loan, artifact, exhibition and condition-report files. Applications must manually match identifiers and synchronise duplicated descriptions. Loan expiry dates copied into a gallery spreadsheet can become inconsistent. Finding artifacts with no condition report requires comparing files and handling unmatched rows manually. The approach can be simple for small lists but is weak for concurrent updates, shared identity and historical integrity.

10.2 Hierarchical model



```
    └── PLACEMENT
        └── ARTIFACT (reference/copy required)
```

A hierarchical tree gives each child one parent path. It supports navigation from lender to loan to artifact, but one artifact may have successive loans and condition reports, and it may be placed in different exhibitions over time. Reusing the artifact under several branches may require duplication or pointer-like mechanisms. A person who is both lender and curator is also awkward to represent as one identity across separate trees.

10.3 Network model

```
LENDER ── owner/member set ── LOAN
LOAN ── owner/member set ── LOAN_OBJECT
ARTIFACT ── owner/member set ── LOAN_OBJECT
ARTIFACT ── owner/member set ── CONDITION_REPORT
EXHIBITION ── owner/member set ── DISPLAY_SLOT
```

A network model supports multiple owner–member paths. An artifact can be reached through several loan occurrences and condition reports. It is more flexible than a strict tree, but applications must understand record pointers and set membership. As the number of relationships grows, navigational code can become complex.

10.4 Query-path comparison

Query	File-based	Hierarchical	Network	Relational DBMS
Find overdue open loans	Scan loan file and filter dates	Traverse each lender's loan children	Navigate lender-to-loan set and test dates	SQL WHERE ReturnDate IS NULL AND DueDate < CURRENT_DATE
Artifacts with no report	Compare artifact and report files	Traverse artifact branch and find missing child	Follow artifact-to-report set and detect empty membership	LEFT JOIN CONDITION_REPORT and filter NULL

10.5 Three-schema architecture

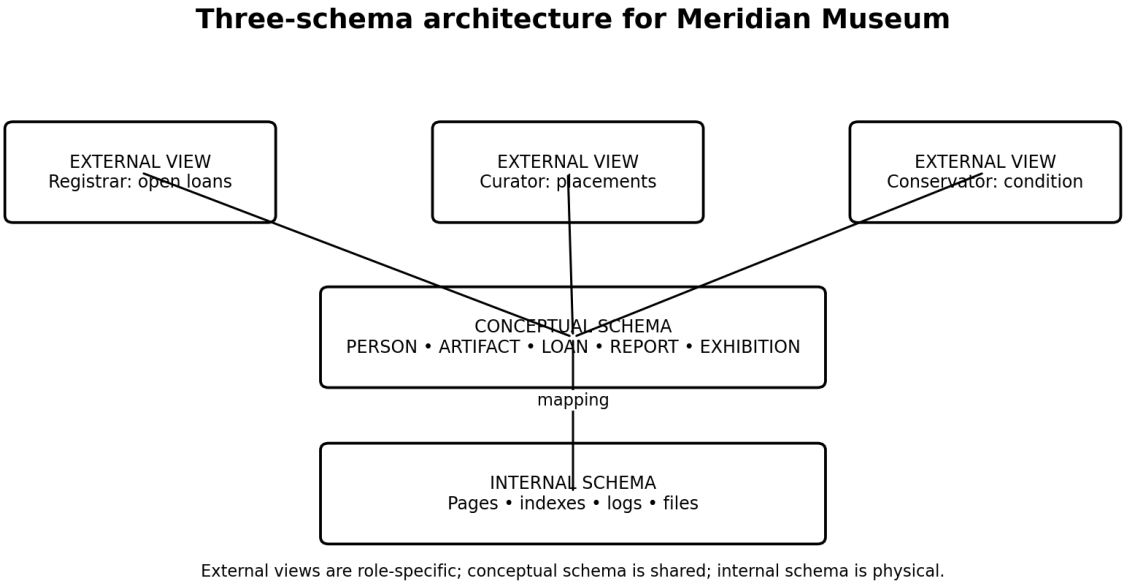


Figure 5. External, conceptual and internal schema layers.

External schemas provide role-specific views: the Registrar sees open loans and returns; the Curator sees exhibition placements; the Conservator sees condition history. The conceptual schema is the shared logical model of PERSON, ARTIFACT, LOAN, LOAN_OBJECT, CONDITION_REPORT, EXHIBITION, DISPLAY_SLOT and PLACEMENT. The internal schema describes physical storage, pages, indexes, record formats and logs.

10.6 Concurrency and transaction management

When two users update the same loan, the transaction manager coordinates the transaction. The concurrency-control or lock manager prevents conflicting updates. The buffer manager manages cached pages, while the log and recovery manager records changes for rollback and crash recovery. The integrity-constraint mechanism checks keys, foreign keys and CHECK rules. A transaction should update loan dates and related custody evidence atomically so that users do not observe a partly updated state.

TASK 11 – RECOMMEND AND DEFEND THE FINAL DESIGN (4 MARKS)

The recommended original design is Alternative B combined with Alternative C: an independently identified LOAN occurrence, LOAN_OBJECT as the loan–artifact associative entity, PERSON as shared identity, and role/subtype records for lender, curator and registrar. PLACEMENT and CONDITION_REPORT remain separate paths because where an artifact is displayed is not the same fact as the object's condition.

11.1 Requirement-to-model traceability

Requirement	Model element	How it is satisfied
One responsible lender per loan	LOAN.LenderPersonID FK	Each loan points to one lender role
Several artifacts per loan	LOAN_OBJECT	One row per artifact line
Successive loans for an artifact	LOAN_OBJECT history	Same ArtifactID may appear under different LoanIDs
Condition evidence by physical object	CONDITION_REPORT	Report references ArtifactID and relevant loan line
Slots numbered within exhibition	DISPLAY_SLOT composite PK	SlotNo is unique only within ExhibitionID
Curator may also be lender	PERSON plus role tables	One identity can hold multiple roles
Legal ownership differs from custody	Ownership history added in Task 12	Loan records do not define legal shares

11.2 Evidence, limitations and scale

DDL constraints can demonstrate row-level validity, uniqueness and referential integrity when executed. Query outputs and performance timings

must be obtained from the actual MySQL run; illustrative panels in this report are not execution proof. Cross-row rules—no overlapping active loans for one artifact, at least one line per confirmed loan, no overlapping placements and ownership shares totalling 100%—require transaction-level validation or controlled procedures.

At 100,000 lenders, numeric surrogate keys, indexed foreign keys, selective indexes on due dates and artifact references, and batch loading support efficient access. Indexes improve lookups but increase storage and write cost. The design should be selected because it represents the museum's real-world distinctions correctly, not because it has a particular number of tables.

TASK 12 – RE-EVALUATE AND ADAPT THE DESIGN (8 MARKS)

12.A Impact analysis (2 marks)

The changed requirement invalidates the assumption that one current lender can be treated as the sole legal owner of an artifact. A lender is the party responsible for a loan or custody agreement; a legal owner is a person or organisation holding an ownership share. These roles may overlap, but they are not equivalent. One lender may coordinate a loan on behalf of several co-owners, and one co-owner may not be the operational lender.

The affected structures are ARTIFACT, PERSON/ORGANIZATION, LENDER_ROLE, LOAN and the ownership association. The old single LenderPersonID cannot describe multiple owners, percentage shares, effective dates or transfers. LOAN.LenderPersonID remains valid for custody responsibility but must not be interpreted as the complete ownership record.

At the external level, the operational view should show the responsible lender and, separately, owners and shares effective on a selected date. At the conceptual level, ARTIFACT_OWNER and OWNERSHIP_PERIOD are

added. At the internal level, indexes are needed for ArtifactID and effective-date access. Existing loan dates and condition reports remain unchanged.

12.B Evaluation of the original design (2 marks)

The EXHIBITION representation is not invalidated. Exhibition and DISPLAY_SLOT continue to describe where an artifact is shown. The failed assumption is the one-to-one interpretation of legal ownership. Exhibition placement cannot be used to infer ownership, and a display slot cannot identify a co-owner.

Stable primary keys—ArtifactID, PersonID, LoanID, ReportID and ExhibitionID—remain valid. The constraint that changes is the assumption that one person is the only legal owner. Temporal constraints are required: ownership shares must be valid over date intervals, conflicting intervals must be prevented, and the total share across owners must equal 100% at each effective point in time. Alternative A still confuses lender custody with ownership. Alternative B remains the strongest custody model but needs a separate ownership association. Alternative C remains useful for shared identity and role overlap, and must be extended to support organisation owners.

12.C Adaptation or redesign (2 marks)

Decision: MODIFY the preferred model rather than replace it. Retain LOAN and LOAN_OBJECT for custody, and add a temporal many-to-many ownership structure. This is the smallest change that represents shared ownership while preserving historical custody evidence.

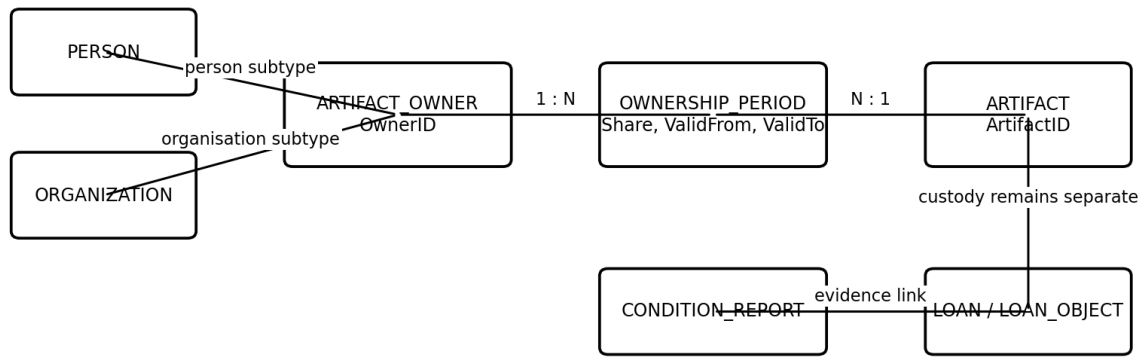


Figure 6. Revised ownership-history model. Custody and condition evidence remain separate.

12.C.1 Revised relational schema

```
ORGANIZATION(
  OrganizationID PK,
  LegalName,
  RegistrationNo UNIQUE
)

ARTIFACT_OWNER(
  OwnerID PK,
  OwnerType,
  PersonID FK NULL,
  OrganizationID FK NULL,
  CHECK (exactly one of PersonID or OrganizationID is non-NULL)
)

OWNERSHIP_PERIOD(
  OwnershipPeriodID PK,
  ArtifactID FK,
  OwnerID FK,
  SharePercent DECIMAL(5,2),
  ValidFrom DATE,
```

```
    ValidTo DATE NULL,
    UNIQUE (ArtifactID, OwnerID, ValidFrom),
    CHECK (SharePercent > 0 AND SharePercent <= 100),
    CHECK (ValidTo IS NULL OR ValidTo >= ValidFrom)
)
```

A more normalised implementation can use ARTIFACT_PERSON_OWNER and ARTIFACT_ORG_OWNER subtype tables under ARTIFACT_OWNER. A transaction procedure or trigger should validate that active ownership shares total 100% and that effective periods do not overlap incorrectly. These are cross-row temporal rules and cannot be guaranteed by row-level CHECK constraints alone.

12.C.2 Migration plan

- Create PERSON/ORGANIZATION owner identities and the ARTIFACT_OWNER and OWNERSHIP_PERIOD tables.
- For each existing artifact, convert the old single-owner value to a 100% ownership row only if the museum confirms that the old field represented legal ownership. If it represented custody responsibility, obtain verified ownership data instead.
- Preserve existing ARTIFACT, LOAN, LOAN_OBJECT, CONDITION_REPORT and PLACEMENT primary keys and timestamps. Do not rewrite historical loans when ownership changes.
- Validate ownership shares and effective dates, resolve missing or conflicting evidence, and record the migration batch/version.
- Switch operational views to read the responsible lender from LOAN and legal owners from the ownership-history tables.

12.D Evidence-based defence (2 marks)

Under the revised rules, an artifact has one or more ARTIFACT_OWNER participants over time. Each OWNERSHIP_PERIOD row records one owner, one artifact, a share and an effective interval. A loan still has one responsible lender and one or more loan lines. Condition reports remain

attached to the relevant artifact and custody line, so an ownership change does not rewrite condition evidence.

12.D.1 Migration validation query

```
-- Find artifacts whose active ownership shares do not total 100%
SELECT a.ArtifactID,
      COALESCE(SUM(op.SharePercent), 0) AS TotalActiveShare
FROM Artifact AS a
LEFT JOIN Ownership_Period AS op
  ON op.ArtifactID = a.ArtifactID
 AND op.ValidFrom <= '2026-11-01'
 AND (op.ValidTo IS NULL OR op.ValidTo >= '2026-11-01')
GROUP BY a.ArtifactID
HAVING COALESCE(SUM(op.SharePercent), 0) <> 100.00;

-- Find condition reports detached from a valid loan line
SELECT cr.ReportID, cr.LoanID, cr.ArtifactID
FROM ConditionReport AS cr
LEFT JOIN LoanObject AS lo
  ON lo.LoanID = cr.LoanID
 AND lo.ArtifactID = cr.ArtifactID
WHERE lo.LoanID IS NULL;
```

The first query reveals artifacts whose active shares are incomplete, excessive or missing for the selected date. The second detects condition reports whose loan-line relationship is invalid. Both are migration checks; they should return zero rows when the migration is valid.

12.D.2 Effect on generated data and operational views

Generated data must now include owner records and ownership periods. If each artifact has an average of two current owners, the ownership association will contain approximately twice as many current ownership rows as artifacts, plus additional rows for historical share changes. Existing lender, loan, loan-line and condition-report counts need not change. Additional association rows and temporal checks increase storage and index-maintenance cost, but they preserve legal facts correctly.

Existing operational views can preserve their interface by retaining LoanID, Lender, Artifact, StartDate, DueDate and ReturnDate, while adding an optional owner-summary view. The final decision is to modify the design because a separate time-aware ownership association is required to represent shared ownership and changing shares without confusing legal title with temporary custody.

CONCLUSION

The Meridian Museum case requires a model that distinguishes physical identity, legal ownership, temporary custody, exhibition placement and condition evidence. The preferred original design uses PERSON with overlapping roles, independently identified LOAN occurrences, LOAN_OBJECT for loan lines, and separate placement and condition-report records. The design supports successive loans and preserves historical inspection evidence. The changed requirement is addressed by adding ARTIFACT_OWNER and OWNERSHIP_PERIOD while retaining the original custody structure. This extension preserves existing loan history and provides a temporal record of ownership shares.

The schema and query set provide a foundation for implementation and validation. The four workload sizes must be loaded reproducibly with seed 4211. Integrity tests should be executed against a dedicated MySQL database, and actual outputs, error messages and median timings should be captured from the user's server before final submission.